

Predicting Benchmark Scores from Unpaired Model Evaluations

Anonymous

Abstract

Evaluating language models is expensive, so benchmark coverage is sparse and noisy: each model is run on a subset of tasks, on a subset of queries within each task, and each score is a noisy average over those queries. Methods that complete the resulting (model $\times$ task) score matrix use only each cell’s aggregate score, and the response-based Data Kernel Perspective Space (DKPS) requires every model to answer the same queries. In this paper, we introduce the **Product Kernel Perspective Space (PKPS)**, which embeds each model from its responses by comparing responses to similar—rather than only identical—queries. PKPS contains DKPS as a special case and remains defined when models answer entirely different queries. Theoretically, we show that PKPS is query-efficient independently of the fraction of queries shared across models. Empirically, on two heterogeneous suites, ensembling the PKPS embedding with the available score signal—a cell’s own sample mean where measured, matrix completion where not—matches or beats the best score-only predictor in every observation regime, with the largest gains where evaluation is sparsest and noisiest. The embedding survives fully unpaired evaluation where DKPS and item-response theory collapse, and rescues small model cohorts where low-rank completion fails.

1 Introduction

Comprehensive evaluation of language models is a growing bottleneck: leaderboards track hundreds of models across dozens of benchmarks, yet no model is run on everything—a recent compilation of 84 frontier models on 133 benchmarks finds only 23% of the score matrix filled [Zeng and Papailiopoulos, 2026]. The (model $\times$ task) score matrix is *doubly sparse*: across tasks (a model is evaluated on a subset of benchmarks) and within tasks (evaluation uses a subset of queries to save cost). Predicting the missing entries would let practitioners reuse cached evaluations instead of paying to re-run them [Polo et al., 2024, Vivek et al., 2024, Zeng and Papailiopoulos, 2026].

The Data Kernel Perspective Space (DKPS) [Helm et al., 2026] is a black-box approach: it embeds each model from its query responses, places models in a low-dimensional Euclidean space via classical multidimensional scaling, and regresses benchmark scores in that space. DKPS is query-efficient and competitive with item-response theory. However, it has a structural limitation: the distance between two models is the average squared difference of their responses *to the same queries*, so DKPS requires a *paired* design. Real evaluation data is routinely unpaired: arena-style platforms score each model on whatever conversations its users happen to have [Chiang et al., 2024]; contamination-driven benchmarks refresh their items, so models evaluated in different months answer different questions [White et al., 2024]; and cost-driven subsampling [Perlitz et al., 2024] gives each evaluation run its own item subset. In all of these, DKPS has nothing to compare and the information in the unpaired responses is thrown away.

We introduce the **Product Kernel Perspective Space (PKPS)**: compare responses to *similar* queries, not only identical ones, by weighting each response-pair comparison with a query kernel. This lets every unpaired response contribute and contains DKPS as the identity-query-kernel special case. The broader thesis is that PKPS supplies an item-level *embedding* signal that score-completion baselines ignore, and that combining it with the available item-level *score* signal predicts benchmark performance better than either alone. Our contributions:

- **Methodological and theoretical.** We extend DKPS to the unpaired regime with a product-kernel affinity $k_Q \cdot k_R$ (Section 3): a strict generalization that contains paired DKPS at $k_Q = \delta$

(and as $\sigma \rightarrow 0$), stays defined on disjoint query sets where DKPS is undefined, and interpolates via a single bandwidth chosen per run by leak-free cross-validation. Extending the proof technique of Helm et al. [2026], we give a query-efficiency guarantee independent of the pairing fraction (Theorem 1), state the strict gap over DKPS as a conjecture, and treat the embedding as one term of an embedding- $\oplus$ -score ensemble.

- **Empirical.** We demonstrate utility and robustness to missing responses on the two ends of one estimation problem—query-efficient evaluation ($m > 0$ queries per cell) and matrix completion ($m = 0$)—on a heterogeneous 18-task, 93-model HELM suite, replicated on a 16-task, 45-model suite of modern models from the Every Eval Ever datastore [EvalEval Coalition, 2026] (Sections 4.1–4.2). The embedding survives unpaired queries where DKPS and IRT collapse and recovers the small-cohort regime where low-rank completion fails; the ensemble matches or beats the best score-only predictor across every lever, by a margin that grows as evaluation gets sparser and noisier.

2 Related work

Low-dimensional representations of models. Some methods compare models through their internals: activations on shared inputs [Duderstadt et al., 2023, Huh et al., 2024, Horwitz et al., 2025] or weights [Chen et al., 2025]. Evaluation, however, is black-box: it sees responses, not internals. The Data Kernel Perspective Space (DKPS) embeds models from their responses via multidimensional scaling [Torgerson, 1952, Helm et al., 2024]; the representations are consistent and concentrate [Acharyya et al., 2024, 2025], support statistical inference [Helm et al., 2025], and are query-efficient [Helm et al., 2026]. But DKPS averages over responses to *shared* queries—a paired design. PKPS keeps the response-level representation and replaces identity matching with a product kernel, so models that answer different queries remain comparable.

Benchmark prediction. A second line predicts unobserved (model, task) scores so that not every evaluation must be run; efficient benchmarking and matrix completion are its two ends. Efficient benchmarking scores a small item subset and extrapolates. Item-response theory selects items by difficulty and discrimination [Rasch, 1960, Polo et al., 2024, Kipnis et al., 2024]; anchor points cluster items by cross-model agreement [Vivek et al., 2024]; cognitive-scale embeddings [Bean et al., 2025] and reinforcement learning [Li et al., 2024] select items adaptively; and design analyses optimize the cost–reliability tradeoff [Perlitz et al., 2024]. Closer to us, DKPS predicts a held-out score from cached responses [Helm et al., 2026], datamodels predict outputs from inputs [Ilyas et al., 2022], and lifelong benchmarks reuse past evaluations [Prabhu et al., 2024]. Matrix completion instead predicts cells that were never run, exploiting low rank across the score matrix [Candès and Recht, 2009, Mazumder et al., 2010, Koren et al., 2009]; BenchPress finds it is approximately rank-two and completes it in logit space [Zeng and Papailiopoulos, 2026]. These methods use only each cell’s *aggregate* score, not its responses. PKPS unifies the two ends—a cell with $m \geq 0$ queries spans efficient benchmarking ($m > 0$) and completion ($m = 0$)—and adds an item-level embedding signal that we ensemble with matrix completion.

3 The Product Kernel Perspective Space

Models and observations. A *model* is a random mapping $f : \mathcal{Q} \rightarrow \mathcal{X}$ from a query space $\mathcal{Q}$ to a response space $\mathcal{X}$; querying f at q returns a response drawn from the distribution $f(q)$. We observe n models $f_1, \dots, f_n$, each run on its own *query set* $Q_i \subseteq \mathcal{Q}$, giving a tuple $\mathbf{Q} = (Q_1, \dots, Q_n)$ that may differ across models or be disjoint; the paired design $Q_1 = \dots = Q_n$ is the special case. Two fixed, distinct embedding functions map raw objects to vectors: a *response* embedding $\text{embed}_R : \mathcal{X} \rightarrow \mathbb{R}^p$

and a *query* embedding $\text{embed}_Q : \mathcal{Q} \rightarrow \mathbb{R}^{p'}$ (e.g. different text-embedding models). For $q_j \in Q_i$ we average the embeddings of the r sampled responses, $x_{ij} = \frac{1}{r} \sum_{k=1}^r \text{embed}_R(f_i(q_j)^{(k)}) \in \mathbb{R}^p$, and write $u_j = \text{embed}_Q(q_j)$ for the embedded query. The goal is to predict each model’s true score on each task, y_{it} —both tasks run with few queries (denoising) and tasks never run (completion); we write y_i when the benchmark has a single score.

Perspective space. Each method summarizes the n models in an $n \times n$ affinity matrix A (defined below). The squared distance between two models is $D^2(a, b) = A_{aa} + A_{bb} - 2A_{ab}$ (clamped at 0), and their d -dimensional *representations* are the classical-multidimensional-scaling minimizer

$$(\hat{\psi}_1, \dots, \hat{\psi}_n) = \arg \min_{z_1, \dots, z_n \in \mathbb{R}^d} \sum_{i,k=1}^n (\|z_i - z_k\| - D(i, k))^2, \quad (1)$$

so model i is the point $\hat{\psi}_i \in \mathbb{R}^d$, its *perspective*. Paired DKPS and PKPS differ only in the affinity A .

Paired DKPS. With all n models answering the same m queries ($Q_1 = \dots = Q_n$), DKPS uses the identity-matched affinity $A_{ab} = \frac{1}{m} \sum_j k_R(x_{aj}, x_{bj})$: response j of model a is paired only with response j of model b , so the design must be paired.

Product kernel. PKPS replaces identity matching with a query kernel k_Q :

$$A_{ab} = \frac{\sum_{j \in Q_a} \sum_{l \in Q_b} k_Q(q_j, q_l) k_R(x_{aj}, x_{bl})}{\sum_{j \in Q_a} \sum_{l \in Q_b} k_Q(q_j, q_l)}, \quad (2)$$

where Q_a is the query set of model a . Each term is normalized by its own query-kernel mass, so models that answered different numbers of (or entirely different) queries are still comparable; responses to *similar* queries—even across tasks—contribute and unpaired evaluations are used (Figure 1). Paired DKPS is the special case $k_Q = \delta$, where only $j = l$ survives.

Kernels. The response kernel k_R compares embedded responses, the query kernel k_Q compares embedded queries. Paired DKPS uses a linear response kernel and the identity query kernel $k_Q = \delta$. We use an RBF query kernel over the embedded queries, $k_Q(q_j, q_l) = \exp(-\|u_j - u_l\|^2 / 2\sigma^2)$, with bandwidth σ chosen per run (below); k_R is RBF in the synthetic study and linear on real data, where linearity makes the kernel factorize across response domains (Section 4.2). Since Eq. 2 normalizes each pair by its own kernel mass, A need not be positive semidefinite; classical MDS embeds the doubly-centered matrix by its top- d nonnegative eigenvalues, as is standard. A_{ab} costs $|Q_a||Q_b|$ kernel terms per pair, and one response-kernel pass serves the whole bandwidth grid.

3.1 Theoretical properties of the PKPS

PKPS is a strict generalization of DKPS.

Proposition 1 (Strict generalization). (i) DKPS limit. With $k_Q = \delta$, Eq. 2 reduces to paired DKPS over the shared queries, $A_{ab} = \frac{1}{|Q_a \cap Q_b|} \sum_{q \in Q_a \cap Q_b} k_R(x_{aq}, x_{bq})$; if the query embeddings are distinct, the RBF query kernel converges to δ as $\sigma \rightarrow 0$, so DKPS is also the small-bandwidth limit. (ii) Unpaired comparability. For any strictly positive k_Q (e.g. RBF), A_{ab} is defined for every pair of nonempty query sets, including disjoint ones, whereas paired DKPS is undefined when $Q_a \cap Q_b = \emptyset$.

Proof. (i) Take $k_Q(q_j, q_l) = \mathbf{1}[q_j = q_l]$. Since the double sum ranges over $j \in Q_a, l \in Q_b$, a term survives only if the two queries are equal *and* that query lies in both sets, $\mathbf{1}[q_j = q_l] \mathbf{1}[q_j \in Q_a] \mathbf{1}[q_l \in Q_b] = \mathbf{1}[q_j = q_l \in Q_a \cap Q_b]$; the normalizer counts the same terms, giving the stated average (the

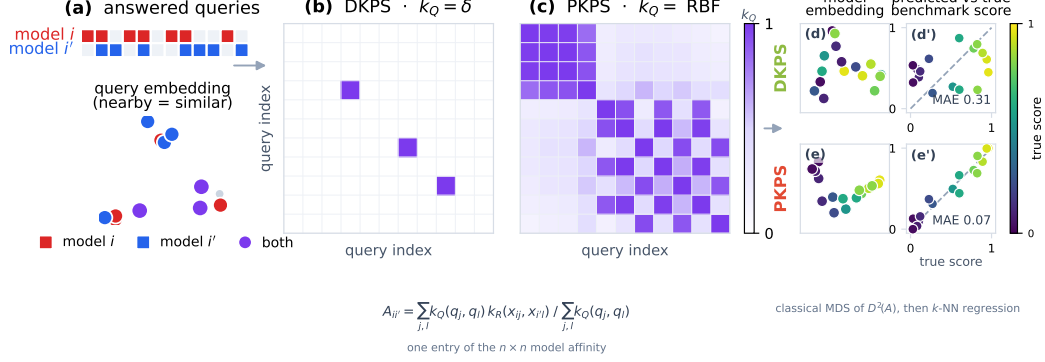


Figure 1: PKPS under *unpaired* evaluation. **(a)** Two models, i (red) and i' (blue), answer their own subsets of a query pool (blocks: who answered what; dots: queries in embedding space, nearby = similar). **(b)** DKPS’s identity kernel keeps only the diagonal cells where both models answered the *same* query—almost every response is discarded. **(c)** An RBF k_Q is a dense similarity matrix: responses to *similar* queries bridge the two models. Both panels index the pooled queries in one shared order; the weights feed one entry of the model affinity (Eq. 2). **(d–e’)** A toy benchmark-prediction example: restricted to the rare shared queries, DKPS’s embedding has no clean score axis and predicts poorly (MAE 0.31); PKPS recovers a smooth score gradient and predicts on the diagonal (MAE 0.07).

same- m -queries case recovers $\frac{1}{m} \sum_j k_R(x_{aj}, x_{bj})$. For the limit, $\exp(-\|u_j - u_l\|^2 / 2\sigma^2) \rightarrow \mathbf{1}[u_j = u_l]$ pointwise as $\sigma \rightarrow 0$. (ii) With $k_Q > 0$ the normalizer $\sum_{j,l} k_Q(q_j, q_l)$ is positive for any nonempty Q_a, Q_b , so the ratio in Eq. 2 is well defined. $\square$

Between the two limits, increasing σ smoothly trades exact query matching for cross-query bridging, so a single scalar controls how much unpaired evidence the perspective uses.

3.2 Inference in the PKPS

Inference is a two-step statistical problem, exactly as in the DKPS [Helm et al., 2026]: embed each model into the perspective space by the scaling of Eq. 1, then regress its benchmark score on the resulting coordinate. Concretely, on a set of reference models with known scores $\{(f_i, y_i)\}$ we fit a decision function $\hat{h} : \mathbb{R}^d \rightarrow \mathcal{Y}$ on the pairs $\{(\hat{\psi}_i, y_i)\}$ and predict any model’s score as $\hat{y}(f) = \hat{h}(\hat{\psi}(f))$. The only free choices are the query-kernel bandwidth σ and the regressor $\hat{h}$, which we describe next; we then add the score channel that turns the embedding into an ensemble.

Choosing the bandwidth. The optimal σ depends on how paired the data is: small when query overlap is high (exact matching), large when it is low (bridging). We set it per run by cross-validation over a median-scaled grid $\sigma \in \{0.03, \dots, 3\} \cdot \text{med}$ (median pairwise query distance), selecting the σ whose perspective best predicts the *observed* scores under leave-one-model-out k -NN—never the held-out scores we report, so the choice is not circular. Because the grid reaches near-delta, cross-validation *recovers DKPS* when overlap is high and bridges only when it must.

Estimation pipeline. For each replicate we (i) PCA-reduce response and query embeddings on the *observed* rows only (dimension by the second Zhu–Ghodsí elbow); (ii) form D from Eq. 2; (iii) embed models by classical MDS; (iv) map the embedding to scores with the *same* head as the matrix-completion baseline up to the factor model—logit transform, per-task standardization, two-way (model + task) bias—then k -NN regression of the residual on the embedding, predicting every cell (held-out cells directly, observed cells leave-one-out). Matching the per-task standardization

matters: without it the model offset is dominated by the highest-logit-variance tasks, biasing the embedding on multi-scale suites.

The embedding- $\oplus$ -score ensemble. Each cell combines the embedding with whichever score signal carries independent information. On a *missing* cell that is matrix completion [Zeng and Papailiopoulos, 2026], blended with PKPS by a weight fit by held-out cross-validation on observed scores. On an *observed* cell it is the cell’s own sample mean, shrunk toward the PKPS estimate by the precision weight $\sigma_q^2/(\sigma_q^2 + e_{PKPS}^2)$ —sample noise variance over total disagreement with PKPS—so the sample dominates as queries grow and the embedding as noise grows. Both quantities are read off observed queries ($\hat{p}(1 - \hat{p})/m$; depth-weighted disagreement), so the weight never touches held-out scores and the score baselines see exactly the same information.

3.3 Query efficiency

PKPS is designed to reach a target prediction accuracy from fewer evaluated queries than paired DKPS: by bridging responses to similar queries it uses evidence that DKPS, limited to shared queries, discards. The query-efficiency analysis of Helm et al. [2026] extends to PKPS. Assume (A1) the benchmark score is bounded and γ -Lipschitz on the population perspective space; (A2) the model distribution places mass on every neighborhood, so a near reference neighbor exists; (A3) kernels and embedded responses are bounded; (A4) the *cross-model query-kernel mass* is bounded below, $\kappa = \inf_{a,b} \mathbb{E}[k_Q(u, u')] > 0$ for queries drawn from any two models’ designs; and (A5) the population squared-distance matrix embeds in $\mathbb{R}^d$, with a spectral gap $\lambda_d > 0$ in its centered Gram matrix. (A4) is the population analogue of the overlap ρ : for an RBF k_Q it holds whenever the query *distributions* overlap in embedding space, even if no individual query is shared.

Theorem 1 (Query efficiency of PKPS). *Under (A1)–(A5), for any $\varepsilon > 0$ there exist (n, m) , with m polynomial in $(1/\varepsilon, 1/\kappa, n)$ and independent of the pairing fraction ρ , such that nearest-neighbor regression in the PKPS attains mean squared error at most ε with high probability.*

Proof sketch. (Full proof in Appendix C.) The affinity of Eq. 2 is a ratio of two two-sample U -statistics over the $|Q_a||Q_b|$ query pairs; by Hoeffding’s inequality and the mass bound (A4), $\varepsilon_\infty = \max_{a,b} |A_{ab} - A_{ab}^*| = O(\kappa^{-2}m^{-1/2})$ w.h.p. (Lemma 1). The perturbed Gram matrix then moves the embedded perspectives by at most $25\sqrt{d}n\varepsilon_\infty/\sqrt{\lambda_d}$ row-wise, via Weyl and a Procrustes alignment under the spectral gap (A5) (Lemma 2). Finally, the nearest-neighbor regression error is at most γ times the population distance to the neighbor (A1), which splits into two estimation terms (above) and the population NN distance, made small by (A2); choosing $m = \text{poly}(n, 1/\kappa, 1/\varepsilon)$ drives the MSE below ε , and no step involves ρ . $\square$

Applied to paired DKPS ($k_Q = \delta$), the same argument retains only the ρM_{ij} shared pairs, so DKPS’s guarantee degrades as $\rho \rightarrow 0$ while Theorem 1 does not. The strict separation—a matching lower bound for DKPS—we leave as a conjecture; Section 4.1 is its empirical counterpart.

Conjecture 1 (Strict gap). *Under (A1)–(A4), the number of queries per model that PKPS needs to reach error ε is at most that of paired DKPS, and the gap increases as $\rho \rightarrow 0$; the two are closest in the fully paired limit $\rho = 1$.*

4 Experiments

A real evaluation matrix is governed by four observation levers. The defining one is (i) *pairing*—the fraction ρ of each cell’s queries shared across models, from fully paired ($\rho=1$) to fully unpaired

($\rho=0$); it is the axis DKPS cannot handle, and we sweep it directly in the synthetic study (Section 4.1). The other three set how sparse and noisy the observations are: (ii) *task coverage*—the fraction of (model, task) cells run at all; (iii) *queries per cell*—how many queries a run uses, which sets the per-cell measurement *noise*; and (iv) *cohort size*—the number of models. On real data (Section 4.2) we turn to the two regimes practitioners actually face, both inherently unpaired since each model is run on its own queries: the all-tasks-covered, few-query limit is efficient benchmarking (denoising) and the zero-query limit is benchmark completion—two applications of the same embedding.

4.1 Synthetic data

We generate n models with latent ability vectors v_i and T tasks with latent vectors t_k , scores $v_i \cdot t_k + \epsilon$, and queries drawn near each task center. Each model answers M_{ij} queries per task, of which $m_{i'j}$ are shared with another model (overlap $\rho = m_{i'j}/M_{ij}$); a task is observed with probability p_{task} and a query with probability p_{query} . We predict held-out (model, task) scores and compare DKPS (identity query kernel) against PKPS (RBF), with the score-only sample baseline (predict a held-out cell by its task’s observed mean) as a floor (Figure 2).

PKPS—selecting its bandwidth by the same leak-free cross-validation as on real data—beats DKPS on *every* lever, and both beat the sample floor: bridging similar queries (across tasks as well as within) supplies signal exact-match DKPS cannot reach, so the margin is wide even when mostly paired (a–c). The gap peaks where pairing is scarce: as p_{query} drops (d) DKPS degrades while PKPS stays flat, and sweeping ρ directly (e), DKPS collapses toward the floor as $\rho \rightarrow 0$ while PKPS is flat—it does not rely on pairing.

Panel (f) is the *denoising* form of the same problem: each cell is now *observed* with M_{ij} noisy queries (sample noise $\propto 1/\sqrt{M_{ij}}$) and we predict its true score. The raw sample mean (gray) falls like $1/\sqrt{M_{ij}}$; smoothing those noisy scores over the perspective denoises well below it while measurement noise dominates—PKPS reaches at a single query the accuracy the raw sample needs roughly five queries to match, and does so whether queries are paired or not, while DKPS denoises only when paired (unpaired DKPS, dashed, barely improves). Only once the budget is large is the raw sample precise enough to overtake the embedding’s residual bias. This is the empirical counterpart of Theorem 1 and Conjecture 1.

4.2 Real data

We study the two applications the unpaired embedding enables: making evaluation more *query-efficient* (denoising cheap, few-query runs) and completing *missing* cells. Both are inherently *unpaired*—each model is run on its own queries, the real-data counterpart of the $\rho \rightarrow 0$ limit swept in the synthetic study: DKPS’s identity matching has little to align and IRT’s shared-anchor fit cannot be estimated, while PKPS bridges similar queries. All real-data numbers are means over 16 seeds, with the bandwidth cross-validated per run. The suite is 18 tasks across four datasets with very different response types—MATH (7 math subjects), WMT-14 (5 translation directions), MedQA (medical multiple-choice), and LegalBench (5 legal classification subsets)—on the 93 models evaluated on all four. MATH and WMT have rich free-text responses; MedQA and LegalBench responses are single tokens for which text embeddings are degenerate, so we encode each dataset’s responses natively (Gemini embeddings vs. one-hot answers) in disjoint blocks. With a linear k_R this makes cross-dataset response similarity exactly zero, and a single shared query embedding with a within-domain bandwidth keeps k_Q near zero across domains: the product kernel factorizes by domain, never comparing a proof to a legal label.

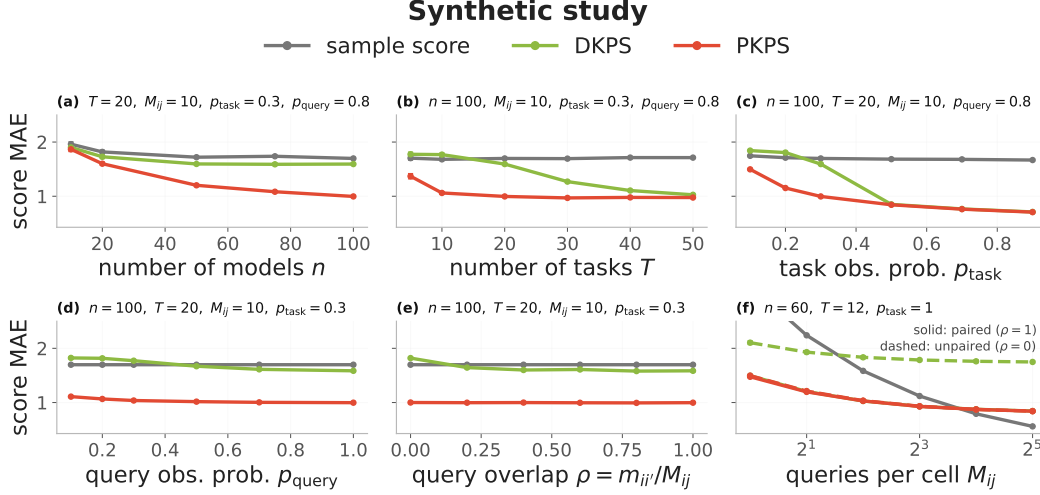


Figure 2: Synthetic study (score MAE, ± 1 SEM over 50 seeds). DKPS (identity query kernel, green) vs. PKPS (RBF, red), with the score-only sample baseline (gray); each panel sweeps one parameter with the rest fixed (shown in its title). (a)–(e) **complete held-out cells** from observed responses—cohort n , tasks T , task coverage p_{task} , query coverage p_{query} , and cross-model overlap $\rho = m_{ij'}/M_{ij}$. PKPS (CV-selected bandwidth) beats DKPS throughout—by a wide margin even when mostly paired (a–c), since it bridges across tasks as well as within—and the gap is largest when unpaired or sparse (d, e), where in (e) DKPS collapses toward the floor as $\rho \rightarrow 0$ while PKPS stays flat. (f) **denoises observed cells**: each cell is measured with M_{ij} noisy queries and we predict its true score, at paired ($\rho=1$, solid) and unpaired ($\rho=0$, dashed). The raw sample falls $\propto 1/\sqrt{M_{ij}}$; smoothing over the perspective denoises well below it at small budgets—PKPS paired or unpaired, DKPS only paired—until the sample becomes precise at the full budget.

Application 1: query efficiency. Because the embedding captures response *style*, which is stable even when the per-cell sample mean is noisy, it denoises cheap evaluations. Each model answers m queries per cell, sampled independently—itsself an unpaired regime, on which IRT is poorly identified; we predict the true full-eval score (Figure 3). At a small budget the embedding is far more accurate than the raw mean: at $m=1$, PKPS scores 0.136 and the ensemble 0.133 versus the sample’s 0.292; PKPS at one query roughly matches the sample at four. As the budget grows the raw mean catches up and eventually overtakes the embedding’s residual bias (0.034 vs. 0.088 at $m=32$); the leak-free ensemble tracks the better channel to within a few thousandths. The advantage is robust across cohort size and task coverage (panels b, c). Per cell, the denoising win is broad but tail-driven: the ensemble beats the raw sample on most cells, with the largest gains at the smallest budgets and on the rich-response datasets (Figure 4).

Application 2: matrix completion. The same embedding also predicts cells that were *never run*. We observe a fraction p_{task} of cells (each at depth p_{query}) and predict the missing ones, against a low-rank matrix-completion baseline (the sample does not apply—a missing cell has no sample). The contrast is the cohort: low-rank completion needs many rows, so when models are scarce it fails for lack of them while the embedding, which needs none, remains accurate—at $n=10$, completion is 0.139 versus PKPS’s 0.120 (Figure 5, dashed). At full cohort ($n=93$, solid) PKPS slightly edges completion (0.103 vs. 0.105) and the ensemble is best (0.099). Even in the regime most favorable to it—a dense, low-rank matrix—completion is matched or slightly beaten; the embedding’s clear advantage is the sparse-cohort regime completion cannot reach. The per-cell win over completion is modest and tail-driven—completion is strong on this low-rank suite—but stays positive in the mean across datasets, cohorts, coverage, and query depth (Figure 6).

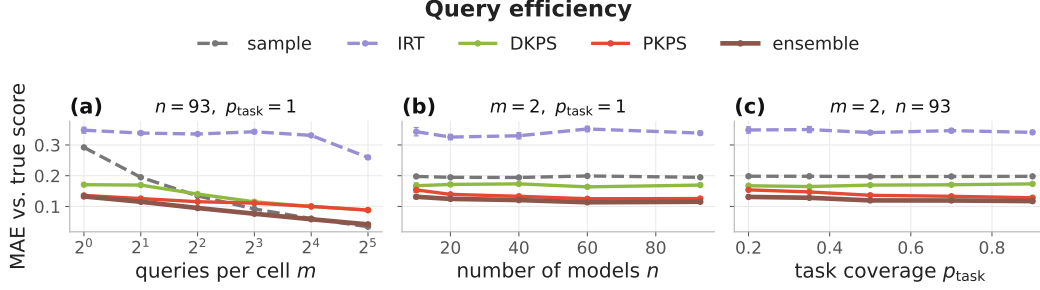


Figure 3: **Query efficiency.** Each model answers m unpaired queries per *observed* cell; predict its true full-eval score (MAE, ± 1 SEM). PKPS (red) and the ensemble (brown) denoise far below the raw sample (grey) at small budgets and across cohort (b) and coverage (c); DKPS (green) trails on unpaired queries, and IRT (purple) is poorly identified without a shared item block.

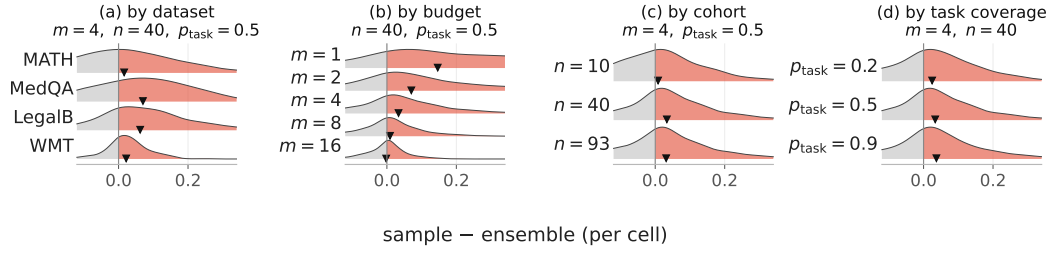


Figure 4: **Conditional breakdown of the query-efficiency win.** Per-cell Δ = sample - ensemble MAE ($\Delta > 0$, red = win) by dataset, budget m , cohort n , and coverage p_{task} , other levers at their sweep median. Broad but tail-driven; the per-row mean ($\blacktriangledown$) is positive throughout.

Second suite: Every Eval Ever. We replicate both applications on a second, independently assembled suite drawn from the Every Eval Ever datastore [EvalEval Coalition, 2026]: 45 modern models on 16 tasks (MATH-MC levels 1–5, GSM-MC, GPQA-Diamond, and nine judging categories from JudgeBench and RewardBench 2), with per-cell query pools drawn independently per model ($\sim 10\%$ incidental overlap)—unpaired by construction. The results mirror the first suite and sharpen the completion contrast (Table 1; lever sweeps in Appendix B): at $m=1$ PKPS attains a third of the sample’s error (0.078 vs. 0.224), and on missing cells the ensemble beats completion at *every* operating point, including full cohort (0.069 vs. 0.096 at $n=45$)—this matrix has higher effective rank, so the response channel wins even where completion is strongest.

Summary. Table 1 collects the two applications on both suites. PKPS is present and competitive in every column, and the ensemble, which folds in whichever score signal exists, is best or tied nearly everywhere.

5 Discussion

Real leaderboards are organic: each model answers whatever queries its authors chose, so the responses benchmark estimation must compare are *unpaired*—and DKPS and item-response models, which need a shared block of identical queries, collapse there. PKPS removes that requirement with a product kernel comparing responses to *similar* queries, recovering paired DKPS in its $\sigma \rightarrow 0$ limit and scaling to heterogeneous suites through a block-diagonal kernel. Our central finding is that benchmark estimation survives unpaired evaluation: as $\rho \rightarrow 0$, DKPS and IRT degrade sharply while PKPS holds. That capability yields two applications from one embedding—denoising cheap, few-query evaluations and predicting never-run cells—and the ensemble with a cell’s own sample is

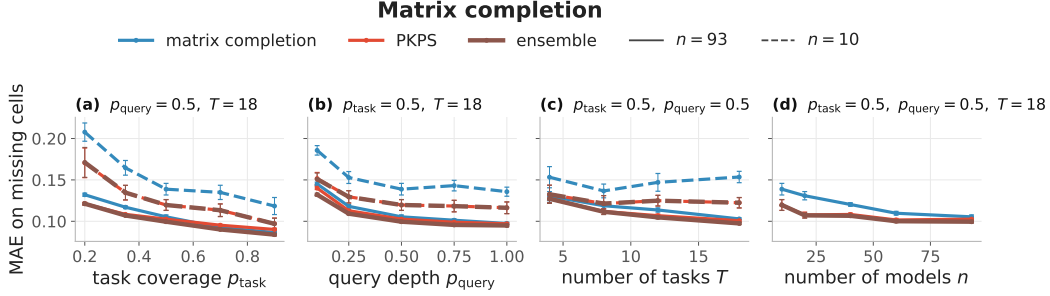


Figure 5: **Matrix completion.** Observe a fraction of (model, task) cells, predict the *missing* ones (MAE on missing cells, ± 1 SEM). Colour is method (low-rank completion blue, PKPS red, ensemble brown); line style is cohort (solid $n=93$, dashed $n=10$). The embedding rescues the small-cohort regime where low-rank completion collapses for lack of rows, and ties it at full cohort.

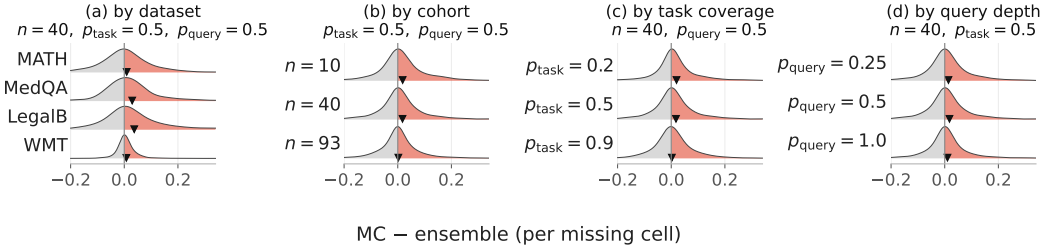


Figure 6: **Conditional breakdown of the completion win.** Per-missing-cell $\Delta = \text{completion} - \text{ensemble MAE}$ ($\Delta > 0$, red = win) by dataset, cohort, coverage, and query depth, other levers at their sweep median. Modest and tail-driven on this low-rank suite, but the per-row mean ($\blacktriangledown$) stays positive.

never the worse choice. None of this assumes pairing or exhaustive evaluation, which organically-assembled leaderboards cannot provide.

Limitations. PKPS embeds every observed response and query; cost and quality inherit from the embedders, which we do not ablate. Our block-diagonal instantiation disables cross-domain bridging; whether a shared response space would help is open. The affinity is quadratic in queries per model pair, needing approximation only for very large query logs. Theorem 1 is existential ((A2) gives no rate in n), and the strict separation from paired DKPS remains a conjecture. Finally, per-cell wins over score-only baselines are modest on low-rank, dense matrices and decisive where evaluation is sparse, noisy, unpaired, small-cohort, or high-rank—the motivating regime.

Table 1: MAE vs. the full-eval score, both suites (16 seeds; best per column bold). Query efficiency at $m \in \{1, 8\}$ queries/cell; completion on missing cells ($p_{\text{task}}=p_{\text{query}}=0.5$) at small and full cohort. “—”: not applicable. The ensemble stays best even where the raw sample catches the embedding ($m=8$), and on the second suite beats completion even at full cohort.

Method	HELM suite (18 tasks, 93 models)				EEE suite (16 tasks, 45 models)			
	Query-eff. ($m=1$)	Query-eff. ($m=8$)	Completion ($n=10$)	Completion ($n=93$)	Query-eff. ($m=1$)	Query-eff. ($m=8$)	Completion ($n=10$)	Completion ($n=45$)
sample mean	0.292	0.092	—	—	0.224	0.081	—	—
IRT	0.348	0.343	—	—	0.165	0.139	—	—
matrix completion	—	—	0.139	0.105	—	—	0.109	0.096
DKPS	0.171	0.115	—	—	0.117	0.083	—	—
PKPS	0.136	0.111	0.120	0.103	0.078	0.059	0.082	0.070
ensemble	0.133	0.076	0.120	0.099	0.087	0.052	0.082	0.069

Reproducibility statement

All methods are released as a documented Python package (the PKPS estimator, matrix completion, and IRT baselines), with the data-preparation, experiment, and figure scripts for both the synthetic study and the HELM suite; every experiment is seeded, all real-data numbers are means over 16 seeds (synthetic: 50), and hyperparameters are selected by the leak-free cross-validation described in Section 3 (leakage is checked by an automated test that corrupts one model’s held-out scores and verifies its predictions are unchanged). Appendix A lists the suite composition, embedding models, and all hyperparameter settings.

References

- Aranyak Acharyya, Michael W. Trosset, Carey E. Priebe, and Hayden S. Helm. Consistent estimation of generative model representations in the data kernel perspective space. *arXiv preprint arXiv:2409.17308*, 2024.
- Aranyak Acharyya, Joshua Agterberg, Youngser Park, and Carey E. Priebe. Concentration bounds on response-based vector embeddings of black-box generative models. *arXiv preprint arXiv:2511.08307*, 2025.
- Andrew M. Bean et al. Scales++: Compute efficient evaluation subset selection with cognitive scales embeddings. *arXiv preprint arXiv:2510.26384*, 2025.
- Emmanuel J. Candès and Benjamin Recht. Exact matrix completion via convex optimization. *Foundations of Computational Mathematics*, 9(6):717–772, 2009.
- Nan Chen, Hayden Helm, Youngser Park, Carey Priebe, and Soledad Villar. Extracting information from fine-tuned weights. In *Non-Euclidean Foundation Models: Advancing AI Beyond Euclidean Frameworks*, 2025.
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios N. Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating llms by human preference. In *International Conference on Machine Learning (ICML)*, 2024.
- Brandon Duderstadt, Hayden S. Helm, and Carey E. Priebe. Comparing foundation models using data kernels. *arXiv preprint arXiv:2305.05126*, 2023.
- EvalEval Coalition. Every eval ever: A unified open data format and public datastore for ai evaluation results. <https://evalevalai.com/projects/every-eval-ever/>, 2026. Dataset: https://huggingface.co/datasets/evaleval/EEE_datastore.
- Hayden Helm, Brandon Duderstadt, Youngser Park, and Carey E. Priebe. Tracking the perspectives of interacting language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- Hayden Helm, Aranyak Acharyya, Youngser Park, Brandon Duderstadt, and Carey Priebe. Statistical inference on black-box generative models in the data kernel perspective space. In *Findings of the Association for Computational Linguistics (ACL)*, 2025.
- Hayden Helm, Ben Johnson, and Carey E. Priebe. Query-efficient model evaluation using cached responses. *arXiv preprint arXiv:2605.07096*, 2026.
- Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
- Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. We should chart an atlas of all the world’s models. *arXiv preprint arXiv:2503.10633*, 2025.

- Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. *arXiv preprint arXiv:2405.07987*, 2024.
- Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. Data-models: Predicting predictions from training data. *arXiv preprint arXiv:2202.00622*, 2022.
- Alex Kipnis, Konstantinos Voudouris, Luca M. Schulze Buschoff, and Eric Schulz. metabench: A sparse benchmark of reasoning and knowledge in large language models. *arXiv preprint arXiv:2407.12844*, 2024.
- Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- Yang Li, Jie Ma, Miguel Ballesteros, Yassine Benajiba, and Graham Horwood. Active evaluation acquisition for efficient llm benchmarking. *arXiv preprint arXiv:2410.05952*, 2024.
- Percy Liang et al. Holistic evaluation of language models. *Transactions on Machine Learning Research (TMLR)*, 2023.
- Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. Spectral regularization algorithms for learning large incomplete matrices. *Journal of Machine Learning Research*, 2010.
- Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking (of language models). In *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinybenchmarks: Evaluating llms with fewer examples. In *International Conference on Machine Learning (ICML)*, 2024.
- Ameya Prabhu, Vishaal Udandara, Philip Torr, Matthias Bethge, Adel Bibi, and Samuel Albanie. Efficient lifelong model evaluation in an era of rapid progress. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, 1960.
- Warren S Torgerson. Multidimensional scaling: I. theory and method. In *Psychometrika*, 1952.
- Stephen Tu, Ross Boczar, Max Simchowitz, Mahdi Soltanolkotabi, and Benjamin Recht. Low-rank solutions of linear matrix equations via procrustes flow. In *International Conference on Machine Learning (ICML)*, 2016.
- Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *EACL*, 2024.
- Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-free llm benchmark. *arXiv preprint arXiv:2406.19314*, 2024.
- Yuchen Zeng and Dimitris Papailiopoulos. You don’t need to run every eval. *arXiv preprint arXiv:2606.24020*, 2026.

A Experimental details

Suite. 18 tasks over four datasets from HELM [Liang et al., 2023]: MATH (7 subjects), WMT-14 (5 translation directions), MedQA, and LegalBench (5 subsets), on the 93 models evaluated on all four; Table 2 lists every task with its query count (the loader subsamples each task to at most 120 queries with a fixed seed, so datasets contribute comparably). MATH and WMT-14 responses are free text and are embedded with gemini-embedding-001 (3072-d); MedQA and LegalBench responses are single-token answers, encoded one-hot. Queries share a single gemini-embedding-001 embedding space. Response and query embeddings are reduced by PCA fit on observed rows only, with dimension chosen by the second Zhu–Ghodsai elbow.

Table 2: HELM suite composition: 18 tasks over four datasets, with the number of unique queries per task. Every task is scored for all 93 shared models; the loader caps each task at 120 queries (seeded subsample).

Dataset	Task	Queries	Dataset	Task	Queries
MATH	algebra	135	WMT-14	cs-en	873
MATH	counting & probability	39	WMT-14	de-en	776
MATH	geometry	38	WMT-14	fr-en	754
MATH	intermediate algebra	52	WMT-14	hi-en	387
MATH	number theory	30	WMT-14	ru-en	613
MATH	prealgebra	86	MedQA	med.qa	1000
MATH	precalculus	57	LegalBench	abercrombie	95
LegalBench	corporate lobbying	490	LegalBench	citizenship questions	1000
LegalBench	function of decision section	367	LegalBench	proa	95

Model coverage (HELM). The 93 shared models span 20 developers: 01-ai-yi-34b, 01-ai-yi-6b, 01-ai-yi-large-preview, AlephAlpha.luminous-base, AlephAlpha.luminous-extended, AlephAlpha.luminous-supreme, ai21.j2-grande, ai21.j2-jumbo, ai21.jamba-1.5-large, ai21.jamba-1.5-mini, ai21.jamba-instruct, allenai.olmo-7b, amazon.nova-lite-v1, amazon.nova-micro-v1, amazon.nova-pro-v1, anthropic.claude-2.0, anthropic.claude-2.1, anthropic.claude-3-5-haiku-20241022, anthropic.claude-3-5-sonnet-20240620, anthropic.claude-3-5-sonnet-20241022, anthropic.claude-3-haiku-20240307, anthropic.claude-3-opus-20240229, anthropic.claude-3-sonnet-20240229, anthropic.claude-instant-1.2, anthropic.claude-instant-v1, anthropic.claude-v1.3, cohere.command, cohere.command-light, cohere.command-r, cohere.command-r-plus, databricks.dbrx-instruct, deepseek-ai.deepseek-llm-67b-chat, deepseek-ai.deepseek-v3, google.gemini-1.0-pro-001, google.gemini-1.0-pro-002, google.gemini-1.5-flash-001, google.gemini-1.5-flash-002, google.gemini-1.5-pro-001, google.gemini-1.5-pro-002, google.gemini-1.5-pro-preview-0409, google.gemini-2.0-flash-exp, google.gemma-2-27b-it, google.gemma-2-9b-it, google.gemma-7b, google.text-bison@001, google.text-unicorn@001, meta.llama-2-13b, meta.llama-2-70b, meta.llama-2-7b, meta.llama-3-70b, meta.llama-3-8b, meta.llama-3.1-405b-instruct-turbo, meta.llama-3.1-70b-instruct-turbo, meta.llama-3.1-8b-instruct-turbo, meta.llama-3.2-11b-vision-instruct-turbo, meta.llama-3.2-90b-vision-instruct-turbo, meta.llama-3.3-70b-instruct-turbo, meta.llama-65b, microsoft.phi-2, microsoft.phi-3-medium-4k-instruct, microsoft.phi-3-small-8k-instruct, mistralai.mistral-7b-instruct-v0.3, mistralai.mistral-7b-v0.1, mistralai.mistral-large-2407, mistralai.mistral-medium-23, mistralai.mixtral-8x22b, mistralai.mixtral-8x7b-32kseqn, mistralai.open-mistral-nemo-2407, nvidia.nemotron-4-340b-instruct, openai.gpt-3.5-turbo-0613, openai.gpt-4-0613, openai.gpt-4-1106-preview, openai.gpt-4-turbo-2024-04-09, openai.gpt-4o-2024-05-13, openai.gpt-4o-2024-08-06, openai.gpt-4o-mini-2024-07-18, openai.text-davinci-002, openai.text-davinci-003, qwen.qwen1.5-110b-chat, qwen.qwen1.5-14b, qwen.qwen1.5-32b, qwen.qwen1.5-72b, qwen.qwen1.5-7b, qwen.qwen2-72b-instruct, qwen.qwen2.5-72b-instruct-turbo, qwen.qwen2.5-7b-instruct-turbo, snowflake.snowflake-arctic-instruct, tiuuai.falcon-40b, tiuuai.falcon-7b, upstage.solar-pro-241126, writer.palmyra-x-004, writer.palmyra-x-v2, writer.palmyra-x-v3.

PKPS. RBF query kernel with bandwidth selected per run by leave-one-model-out k -NN ($k=8$) on the observed scores over the grid $\sigma \in \{0.03, 0.1, 0.3, 1, 3\} \times \text{median pairwise query distance}$, plus the near-delta (DKPS) limit; linear response kernel on real data, RBF on synthetic. Response blocks are capped at 48 PCA components per dataset and the shared query space at 64; each (model, query)

pair uses one sampled response ($r=1$). Models are embedded by classical MDS ($d=12$ on real data, fixed a priori and not tuned; d = latent dimension on synthetic).

Prediction head. All embedding methods share one head: logit-transformed scores, per-task standardization to unit variance, a two-way (model + task) additive bias, and a distance-weighted k -NN ($k=5$) regression of the residual on the model embedding. Held-out cells are predicted directly; observed cells leave-one-out.

Baselines. Matrix completion follows BenchPress [Zeng and Papailiopoulos, 2026]: logit space, per-task standardization, two-way bias, rank-2 ALS with ridge $\lambda=0.1$, averaged over 5 random initializations; on observed cells it is cross-fit so a cell’s prediction never sees its own sample. IRT is a 1PL (Rasch) model on binary tasks: item difficulties by maximum likelihood (L-BFGS) on the largest fully-observed (model $\times$ item) block, per-model ability by bounded scalar MLE, clipped to $[-4, 4]$. The sample baseline is the cell’s own m -query mean.

Ensemble. On observed cells, the sample mean is shrunk toward the PKPS estimate with per-cell precision weight $e_{PKPS}^2 / (e_{PKPS}^2 + \hat{p}(1 - \hat{p})/m)$ on the sample, where e_{PKPS}^2 is the embedding’s depth-weighted disagreement with the observed samples of that model; on missing cells, PKPS and matrix completion are blended with a weight fit by held-out cross-validation against observed scores. No quantity ever uses the held-out full-eval scores.

Sweeps and reporting. All real-data sweeps use 16 seeds; curves and table entries report the per-seed mean over tasks, then the mean (± 1 SEM) over seeds. Query efficiency: budgets $m \in \{1, 2, 4, 8, 16, 32\}$ at full cohort and coverage; cohort $n \in \{10, 20, 40, 60, 93\}$ (HELM) or $\{10, 20, 30, 45\}$ (EEE) and task coverage $p_{\text{task}} \in \{0.2, 0.35, 0.5, 0.7, 0.9\}$, both at the few-query point $m=2$. Completion: task coverage as above and query depth $p_{\text{query}} \in \{0.1, 0.25, 0.5, 0.75, 1.0\}$ with cohort lines $n \in \{10, \text{full}\}$; task count $T \in \{4, 8, 12, 18\}$ (HELM) or $\{4, 8, 12, 16\}$ (EEE); the cohort sweep holds $p_{\text{task}}=p_{\text{query}}=0.5$.

Synthetic study. 50 seeds per configuration; latent dimension 5, observed embedding dimension 20, score noise 0.1, response noise 0.5; both kernels RBF; k -NN regression with $k=5$. Panel-specific settings are given in each figure title.

Compute. All experiments run on a single multi-core CPU node; text embeddings are precomputed once through the embedding API and cached. Embedding the EEE suite (30,304 pooled responses plus 7,665 queries, ~ 11 M tokens) took ~ 13 minutes and under \$2 of API credit; each 16-seed sweep completes in minutes (EEE) to hours (HELM) on the same node.

B The Every Eval Ever suite

The second suite is built from the Every Eval Ever datastore [EvalEval Coalition, 2026], an MIT-licensed, community-maintained collection of item-level evaluation results in a unified format (inputs, raw model outputs, and per-item scores). We take the five benchmarks with the widest item-level model coverage—MATH-MC, GSM-MC, GPQA-Diamond, JudgeBench, and RewardBench 2—restricted to the 45 models present on all five; sub-tasks (MATH-MC levels 1–5; four JudgeBench and five RewardBench 2 categories) give 16 tasks (Table 3). Every domain is free text (including the judging benchmarks’ rationales), so all responses and queries are embedded with

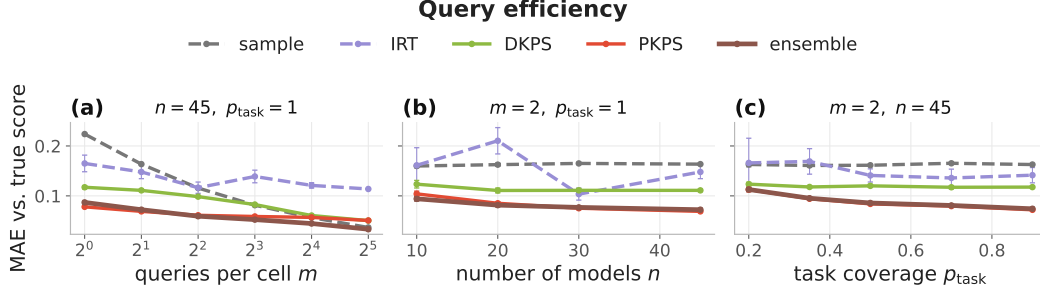


Figure 7: **Query efficiency on the Every Eval Ever suite** (45 models, 16 tasks; MAE vs. full-depth score, ± 1 SEM over 16 seeds). Same protocol as Figure 3. PKPS at $m=1$ attains a third of the sample’s error and the sample needs $m \approx 8$ to catch up; DKPS trails because pools are unpaired, and IRT is poorly identified. The ensemble is best from $m \geq 2$ on.

gemini-embedding-001 and blocked by benchmark exactly as in Appendix A. Each cell’s observable pool is 32 queries drawn with a model-dependent seed, so two models share only incidental queries ($\sim 10\%$); the query-efficiency budget m and the completion depth p_{query} subsample this pool, and the prediction target is always the cell’s full-depth mean over all items, which is never embedded. All scores are binary, so IRT applies to every task (on the HELM suite it excludes WMT). Figures 7 and 8 repeat the lever sweeps of Figures 3 and 5 on this suite; Table 1 summarizes.

Table 3: EEE suite composition: 16 tasks over five benchmarks, with the number of items per (model, task) cell. Every task is fully answered by all 45 shared models; each cell’s observable pool is 32 items, sampled per model.

Benchmark	Task	Items	Benchmark	Task	Items
MATH-MC	level 1	430	JudgeBench	coding	42
MATH-MC	level 2	882	JudgeBench	knowledge	154
MATH-MC	level 3	1115	JudgeBench	math	56
MATH-MC	level 4	1199	JudgeBench	reasoning	98
MATH-MC	level 5	1288	RewardBench 2	factuality	475
GSM-MC	gsm_mc	1319	RewardBench 2	focus	495
GPQA-Diamond	gpqa_diamond	198	RewardBench 2	math	183
RewardBench 2	precise if	160	RewardBench 2	safety	450

Model coverage (EEE). The 45 models shared by all five benchmarks span 13 developers: cohere/c4ai-command-a-03-2025, cohere/c4ai-command-r-08-2024, cohere/c4ai-command-r-plus-08-2024, cohere/c4ai-command-r7b-12-2024, cohere/command-a-reasoning-08-2025, deepseek/deepseek-r1-0528, deepseek/deepseek-v3-1-terminus, deepseek/deepseek-v3-2, deepseek/deepseek-v3-2-speciale, deepseek/deepseek-v4-flash-fp8, deepseek/deepseek-v4-pro, google/gemini-3-1-pro-preview, google/gemma-2-27b-it, google/gemma-2-9b-it, google/gemma-3-12b-it, google/gemma-3-27b-it, google/gemma-4-e2b-it, google/gemma-4-e4b-it, llm360/k2-v2-instruct, meta/llama-4-maverick-17b-1, meta/meta-llama-3-1-8b-instruct, minimax/minimax-m2-5, mistral/mistral-small-4-119b-2603, moonshot/kimi-k2-5, moonshot/kimi-k2-6, moonshot/kimi-k2-thinking, openai/gpt-5-5, openai/gpt-5-mini, openai/gpt-5-nano, openai/gpt-oss-120b, openai/gpt-oss-20b, qwen/qwen3-30b-a3b-thinking-2507, qwen/qwen3-5-122b-a10b, qwen/qwen3-5-2b, qwen/qwen3-5-397b-a17b, qwen/qwen3-6-35b-a3b, qwen/qwen3-6-plus, qwen/qwen3-next-80b-a3b-thinking, stepfun/step-3-5-flash, xiaomi/mimo-v2-flash, zai-org/glm-4-6-fp8, zai-org/glm-4-7-flash, zai-org/glm-4-7-fp8, zai-org/glm-5-1-fp8, zai-org/glm-5-fp8.

C Proof of Theorem 1

Population quantities. Queries for model a are drawn i.i.d. from a design distribution over embedded queries; write $z = (u, x_a(u))$ for an embedded query–response pair (response noise included)

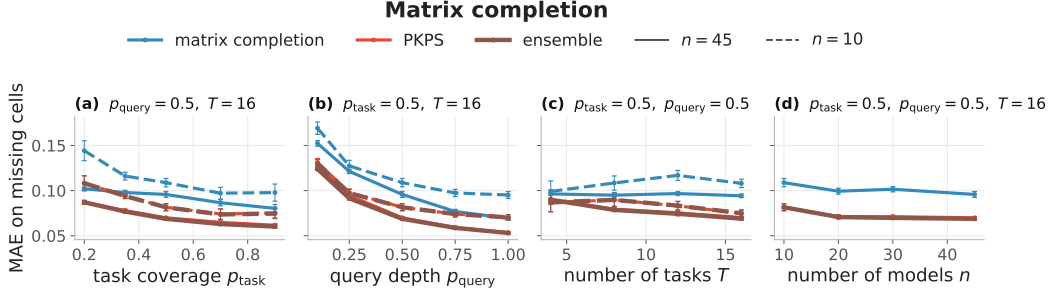


Figure 8: **Matrix completion on the Every Eval Ever suite** (MAE on missing cells, ± 1 SEM; solid $n=45$, dashed $n=10$). Same protocol as Figure 5. Unlike the HELM suite, the embedding and ensemble beat low-rank completion at every operating point—including full cohort and coverage—consistent with the higher effective rank of this modern score matrix.

and P_a for its law. By (A3) the kernels are bounded: $k_Q \in [0, 1]$ after normalization (RBF) and $|k_R| \leq B_R$. Define the population affinity

$$A_{ab}^* = \frac{\mathbb{E}[k_Q(u, u') k_R(x_a(u), x_b(u'))]}{\mathbb{E}[k_Q(u, u')]}, \quad z \sim P_a, z' \sim P_b \text{ independent},$$

population squared distances $D^{*2}(a, b) = A_{aa}^* + A_{bb}^* - 2A_{ab}^*$, the centered Gram matrix $B^* = -\frac{1}{2}JD^{*2}J$ with $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, and population perspectives $\Psi^* \in \mathbb{R}^{n \times d}$ its rank- d spectral embedding, which exists with $\lambda_d(B^*) > 0$ by (A5).

Lemma 1 (Affinity concentration). *Under (A3)–(A4), for models a, b with $\min(|Q_a|, |Q_b|) \geq m$ and any $\delta \in (0, 1)$: with probability at least $1 - \delta$,*

$$|A_{ab} - A_{ab}^*| \leq \frac{2(1+B_R)}{\kappa^2} t_m(\delta), \quad t_m(\delta) = \bar{B} \sqrt{\frac{\log(4/\delta)}{2m}}, \quad \bar{B} = \max(2B_R, 1),$$

provided $t_m(\delta) \leq \kappa/2$.

Proof. The numerator $N = \frac{1}{|Q_a||Q_b|} \sum_{j,l} k_Q(u_j, u'_l) k_R(x_{aj}, x_{bl})$ and denominator $Z = \frac{1}{|Q_a||Q_b|} \sum_{j,l} k_Q(u_j, u'_l)$ of Eq. 2 are two-sample U -statistics of order $(1, 1)$ in the i.i.d. pairs $z_j \sim P_a, z'_l \sim P_b$, with kernels of range at most $2B_R$ and 1 respectively (response noise enters only through the bounded kernel argument, so the number of responses per query may stay fixed). By Hoeffding’s inequality for two-sample U -statistics [Hoeffding, 1963, Section 5], each deviates from its mean by more than its range times $\sqrt{\log(4/\delta)/2m}$ with probability at most $\delta/2$; since $\bar{B}$ dominates both ranges, a union bound gives $|N - N^*| \leq t_m(\delta)$ and $|Z - Z^*| \leq t_m(\delta)$ jointly with probability $\geq 1 - \delta$. On this event, since $Z^* \geq \kappa$ by (A4) and $t_m(\delta) \leq \kappa/2$, we have $Z \geq \kappa/2$, and

$$|A_{ab} - A_{ab}^*| = \frac{|NZ^* - N^*Z|}{ZZ^*} \leq \frac{|N - N^*|}{Z} + \frac{|N^*||Z - Z^*|}{ZZ^*} \leq \frac{2t_m(\delta)}{\kappa} + \frac{2B_R t_m(\delta)}{\kappa^2} \leq \frac{2(1+B_R)}{\kappa^2} t_m(\delta),$$

using $|N^*| \leq B_R$ and $\kappa \leq 1$. $\square$

Lemma 2 (Perspective perturbation). *Let $\varepsilon_\infty = \max_{a,b} |A_{ab} - A_{ab}^*|$. There is an orthogonal $W \in \mathbb{R}^{d \times d}$ such that*

$$\max_i \|\hat{\psi}_i - W\psi_i\| \leq \frac{25\sqrt{d} n \varepsilon_\infty}{\sqrt{\lambda_d(B^*)}}.$$

Proof. Entrywise, $|D^2 - D^{*2}| \leq 4\varepsilon_\infty$, and double centering inflates an entrywise bound by at most a factor of 4, so $E = B - B^*$ satisfies $\|E\|_{\text{op}} \leq \|E\|_F \leq 8n\varepsilon_\infty$. Let B_d be the top- d truncation of B that classical MDS embeds ($B_d = \hat{\Psi}\hat{\Psi}^\top$, positive semidefinite). By Weyl’s inequality every eigenvalue of

B beyond the d -th, and every negative eigenvalue, has magnitude at most $\|E\|_{\text{op}}$ (since $\lambda_{d+1}(B^*) = 0$ and $B^* \succeq 0$), so $\|B_d - B\|_{\text{op}} \leq \|E\|_{\text{op}}$. The matrix $B_d - B^*$ has rank at most $2d$, hence

$$\|B_d - B^*\|_F \leq \sqrt{2d} \|B_d - B^*\|_{\text{op}} \leq \sqrt{2d} (\|B_d - B\|_{\text{op}} + \|E\|_{\text{op}}) \leq 2\sqrt{2d} \|E\|_F.$$

Both $B_d = \hat{\Psi}\hat{\Psi}^\top$ and $B^* = \Psi^*\Psi^{*\top}$ are positive semidefinite of rank at most d , so by the Procrustes alignment lemma of [Tu et al. \[2016, Lemma 5.4\]](#),

$$\min_{W \in O(d)} \|\hat{\Psi} - \Psi^*W\|_F^2 \leq \frac{\|B_d - B^*\|_F^2}{2(\sqrt{2} - 1)\lambda_d(B^*)}.$$

Combining, $\max_i \|\hat{\psi}_i - W\psi_i\| \leq \|\hat{\Psi} - \Psi^*W\|_F \leq \frac{2\sqrt{2d}}{\sqrt{2}(\sqrt{2}-1)} \cdot \frac{8n\epsilon_\infty}{\sqrt{\lambda_d(B^*)}} \leq \frac{25\sqrt{d}n\epsilon_\infty}{\sqrt{\lambda_d(B^*)}}$. Nearest-neighbor distances are invariant to W . $\square$

Proof of Theorem 1. Take scores in $[0, 1]$ without loss of generality (A1). Fix $\epsilon > 0$ and set targets $c \leq \sqrt{\epsilon/2}/(8\gamma)$ for the estimation error and $\epsilon' \leq \sqrt{\epsilon/2}/(2\gamma)$ for the population nearest-neighbor distance. By (A2) there is $n_0(\epsilon')$ such that for $n \geq n_0$, a reference model within population distance ϵ' of the target exists except with probability at most $\epsilon/8$. Fix such an n . Applying Lemma 1 with confidence $\delta = \epsilon/(8n^2)$ to every pair, a union bound and Lemma 2 give $\max_i \|\hat{\psi}_i - W\psi_i\| \leq c$ except with probability at most $\epsilon/8$, once

$$m \geq C' \frac{d(1+B_R)^2 \bar{B}^2 n^2 \log(n/\epsilon)}{\kappa^4 \lambda_d c^2} = \text{poly}(n, 1/\kappa, 1/\epsilon),$$

for an absolute constant C' (this choice also satisfies the proviso $t_m \leq \kappa/2$ of Lemma 1), and does not involve the pairing fraction ρ . On the intersection of the two events, for a target model f with nearest estimated reference neighbor f^* (perspectives $\psi^*, \hat{\psi}^*$; the target's $\psi, \hat{\psi}$),

$$|\hat{y}(f) - y(f)| \leq \gamma \|\psi^* - \psi\| \leq \gamma (\|\hat{\psi}^* - W\psi^*\| + \|\hat{\psi}^* - \hat{\psi}\| + \|\hat{\psi} - W\psi\|) \leq \gamma(2c + \hat{\delta}),$$

using (A1) and orthogonal invariance. The estimated nearest-neighbor distance $\hat{\delta}$ is at most the estimated distance to the ϵ' -close reference, itself at most $\epsilon' + 2c$ by the same alignment argument, so $|\hat{y} - y| \leq \gamma(4c + \epsilon') \leq \sqrt{\epsilon/2}$. Squaring, and adding the two failure events (squared errors are bounded by 1), $\text{MSE}(\hat{y}) \leq \epsilon/2 + \epsilon/4 \leq \epsilon$. $\square$

Contrast with paired DKPS. With $k_Q = \delta$, only identically-shared query pairs enter the double sum, so the U -statistic runs over the ρM_{ij} shared queries and the rate in Lemma 1 becomes $O_P((\rho M_{ij})^{-1/2})$: DKPS's guarantee requires $\rho M_{ij} \rightarrow \infty$ and vacates in the unpaired limit $\rho \rightarrow 0$, where its distance is undefined (Proposition 1). This asymmetry motivates Conjecture 1; a formal lower bound for DKPS is future work.